A

PROJECT REPORT

on

EMAIL PRIORITY AND ACTION REMINDER SYSTEM

Submitted by

SUKANYA KAKE

Under the guidance of

N . SATYANANDARAM

LECTURER, MSIT-IIITH

Department of Computer Science and Engineering

2025-2026

Rajiv Gandhi University of Knowledge and Technologies(RGUKT),

R.K.Valley, Kadapa, Andhra Pradesh, 516 330

`

Rajiv Gandhi University of Knowledge and Technologies (RGUKT),
RK Valley, Kadapa (Dist), Andhra Pradesh, 516330

CERTIFICATE

This is to certify that the project work titled **"EMAIL PRIORITY AND ACTION REMINDER SYSTEM "** is a bonafied project work submitted by K**. SUKANYA (R200633)** in the department of COMPUTER SCIENCE AND ENGINEERING in partial fulfillment of requirements for the award of degree of Bachelor of Technology in **Computer Science and Engineering** for the year 2025- 2026 carried out the work under the supervision.

GUIDE	**HEAD OF THE DEPARTMENT**
Mr.N.SATYANANDARAM	Dr.CH.RATNA KUMARI
LECTURER (MSIT-IIITH)	ASSISTANT PROFESSOR(M Tech(UoH))
DEPARTMENT OF CSE	DEPARTMENT OF CSE

EXTERNAL EXAMINER

`

DECLARATION

This is Title: EMAIL PRIORITY & ACTION REMINDER SYSTEM

-- ActionFirst Inbox

Degree for which the Thesis is submitted: Bachelors of Technology in Computer Science Engineering.

I declare that the presented thesis represents largely my own ideas and work in my own words. Where other's ideas or words have been included, I have adequately cited and listed in the reference materials. The thesis has been prepared without resorting to plagiarism. I have adhered to all principles of academic honesty and integrity. No falsified or fabricated data have been presented in the thesis. I understand that any violation of the above will cause for disciplinary action by the Institute, including revoking the conferred degree, if conferred, and can als o evoke penal action from the sources which have not been properly cited or from whom proper permission has not been taken.

Date : 18 NOVEMBER 2025 **K. SUKANYA (R200633)**

Place : RGUKT RK VALLEY

`

ACKNOWLEDGEMENT

The Satisfaction that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose constant guidance and encouragement crown all the efforts success.

Iam extremely grateful to our respected Director**,** **Prof.A.V.S.S.KUMARA SWAMI GUPTHA** for fostering an excellent academic climate in our institution.

I also express my sincere gratitude to our respected Head of the Department **Dr.CH.RATNA KUMARI** for her encouragement, overall guidance in viewing this project a good asset and effort in bringing out this project.

I would like to convey thanks to our guide at college **Mr.N.SATYANANDARAM** for his guidance, encouragement, co-operation and kindness during the entire duration of the course and academics.

My sincere thanks to all the members who helped me directly and indirectly in the completion of project work. I express my profound gratitude to all our friends and family members for their encouragement.

`

TABLE OF CONTENTS

S.NO	INDEX	PAGE NO
	Abstract	06
	List of Figures	08
1.	Introduction • Problem Statement • Motivation • Contribution	09-15
2.	UI/UX Design	16-18
3.	Modules	
	Module-1: User Authentication	19
	Module-2: Email Triage & Action Management	20
	Module-3: Database and Data Models	22
	Module-4: User Interface	23
	Module-5: Email Communication & Retrieval	24
	Module-6: ML Classification Core	26
	Module-7:Reminder & Scheduling System	27
	Module-8: API & Endpoint Management	28
4.	Results and Discussion	30
5.	Conclusion and Future Enhancements	34
	References	36

`

ABSTRACT

The **ActionFirst Inbox** platform is a web-based application built on **Python** and the **Flask** framework, designed to optimize email management by leveraging machine learning for intelligent prioritization. This system's primary objective is to transform a standard email client into an action-oriented workflow, allowing users to efficiently focus on the messages that matter most based on the predicted level of urgency. The application handles email retrieval by connecting to an **IMAP** server and enables users to send replies via **SMTP**.

The core technology stack relies on **MongoEngine** to interface with a **MongoDB** database, which is used for the persistence of key data models, including **User** accounts, **ClassifiedEmail** documents, and **Reminder** records. User authentication is secured by hashing passwords using the **bcrypt** library before storing them. Critical dependencies enabling the system's functionality include **scikit-learn** for machine learning, **pandas** for data handling, and **IMAPClient** for email fetching.

The machine learning functionality is encapsulated in the **EmailClassifier**, which employs a **RandomForestClassifier** trained on email data. To prepare text for classification, email subjects and bodies are combined, cleaned by lowercasing and removing punctuation, and stripped of common stopwords. The textual content is then converted into a numerical feature set using a **TfidfVectorizer**. The classifier's output dictates the email's **predicted priority**—which can be **HIGH**, **MEDIUM**, or **LOW**—and suggests a next-step **action**, mapping HIGH priority to "Reply," MEDIUM to "Schedule," and LOW to "Ignore".

Upon successful user login, the application performs an initial live fetch of the user's latest emails, applies the machine learning classification, and saves the results. The main **dashboard** allows users to view their active emails, providing

`

options to filter the list by **HIGH**, **MEDIUM**, or **LOW** priority, and supports full-text **search** across subject, sender, and email snippet.

For workflow management, users can **mark an email as done** or **delegate** it, effectively clearing it from their active list. A unique feature is the ability to **set reminders** on any email, which are managed as background tasks by the **APScheduler** to provide a timely notification to the user.

`

LIST OF FIGURES

Figure No.	Title	Page No.
Figure1.1	Complete process	10
Figure2.1	Persona	16
Figure2.2	High Fedality Wireframes for Single Email Detail View	17
Figure2.3	High Fedality Wireframes for Dashboard	18
Figure3.11	User Login	19
Figure3.12	User Register	20
Figure3.21	Filtering Low Priority Emails	21
Figure3.22	Email Detail View	21
Figure3.31	Database	22
Figure3.32	Classified Email	23
Figure3.41	Responsive Design & Filtering	24
Figure3.51	Fetching Emails using IMAP	25
Figure3.52	Replying Emails Through SMAP	25
Figure3.61	ML Flow	26
Figure3.71	Remainders and Notifications	27
Figure4.1	Frontend	32
Figure4.2	Backend	33

LIST OF TABLES

Table No	Title	PageNo
1	User Authentication API	28
2 & 3	Email Triage API,Communication and Scheduling API	29
4	Challenges and Solution	31

`

1. INTRODUCTION

In today's digital era, the massive volume of email poses a significant barrier to productivity and timely communication. With the increasing reliance on digital correspondence, there is a growing need for smart inbox applications that are accessible, secure, and capable of automatically identifying message priority. This project presents **ActionFirst Inbox**—a full-featured, smart email management application designed to solve this problem by automatically prioritizing and directing user action.

The main goal of this project is to create a reliable and intelligent platform where users can immediately identify and act upon their most critical messages. Key features include **Machine Learning classification** of emails into HIGH, MEDIUM, or LOW priority, suggested actions like **Reply**, **Schedule**, or **Ignore**, **live email retrieval** via IMAP, and the ability to set **time-based reminders** for follow-up using the **APScheduler** background task manager. Users can also mark emails as done, delegate tasks, and send replies via SMTP.

ActionFirst Inbox is built using modern web technologies. The back-end is powered by **Python** and the **Flask** web framework, with **MongoDB** (via MongoEngine) used for secure data storage of user profiles and classified emails. Email prioritization is handled by the **scikit-learn** library, specifically utilizing a **RandomForestClassifier** and **TfidfVectorizer**.

User authentication is handled securely using **bcrypt** for password hashing. This project successfully integrates a robust web application with machine learning to create a proactive and efficient inbox experience.

`

```
+-------------------------------------------------------------+
|              A C T I O N F I R S T   I N B O X               |
+-----------+----------------------+---------------------------+
| LOGIN     | ML CLASSIFIER        | USER ACTIONS              |
| • Auth    | • Clean + TF-IDF     | • Mark Done → MongoDB     |
| • IMAP    | • Random Forest Pred | • Reply → SMTP            |
| • Fetch   | • Priority (H/M/L)   | • Reminder → APScheduler  |
+-----------+----------------------+---------------------------+
| SYSTEM FLOW:                                                |
| RAW EMAIL → Preprocess → Vectorize → Predict → Save → UI    |
+-------------------------------------------------------------+
```

Figure1.1: Complete process.

PROBLEM STATEMENT

In today's fast-paced digital world, effectively managing the relentless influx of email is a critical challenge to both personal and professional productivity. Conventional email clients, which present messages chronologically, are no longer sufficient as they fail to **differentiate between urgent, actionable items and low-priority communications** [Implied by the system's purpose]. This lack of automated intelligence forces users to engage in time-consuming manual sorting, leading to decision fatigue, delayed responses, and a reactive workflow.

The **ActionFirst Inbox** project is specifically designed to address the following critical problems inherent in conventional email management:

- **Prioritization and Actionability:** Users struggle to instantly identify the true urgency of an email, leading to wasted time reading low-priority messages. The system must automatically and accurately **classify messages** to suggest the immediate next step, such as **Reply**, **Schedule**, or **Ignore**, based on their predicted priority.

`

- **Timeliness and Follow-up:** Important emails that require action at a future date are often forgotten once they leave the immediate view. The system needs a reliable mechanism to **set time-based alerts** and bring deferred items back to the user's attention using a scheduling system.
- **Security and Access:** Email content and user access must be safeguarded. The platform must implement secure practices for user authentication and connection to external mail servers, which is addressed by using **bcrypt for password hashing** and secure IMAP/SMTP connections.
- **Efficiency and Organization:** The dashboard should allow for easy segmentation of the active workload. The system must support quick **filtering by the predicted priority** (HIGH, MEDIUM, LOW) and provide robust **search capabilities** across email content to find specific messages without unnecessary clutter.
- **Workflow Completion:** There is no simple way to mark an email as a completed task within the inbox interface. The application must provide clear actions like **"Mark Done"** or **"Delegate"** to remove completed items from the active list, ensuring the inbox reflects only current, actionable work.
- **User Experience:** Developing a simple, intuitive interface that ensures users can quickly create, edit, and manage notes without frustration, whether on desktop or mobile devices.

MOTIVATION

A significant focus of the **ActionFirst Inbox** project is to combat the pervasive problem of digital overload by providing an **intelligent and action-driven** email management experience [Implied by the system's purpose]. The platform is designed to be efficient, ensuring that the critical task of email triage is handled seamlessly and automatically, which frees the user to focus on their actual work rather than their inbox.

This report will explore the development lifecycle, from the initial concept and design to the implementation and testing phases of the classification model and integrated workflow. It will highlight the challenges encountered during

`

development, such as ensuring accurate machine learning classification and integrating complex scheduling, and the solutions used to address these challenges. By the end of this report, readers will gain a thorough understanding of the ActionFirst Inbox platform, its features, and the technologies behind its creation.

- **The Need for Automated Prioritization:**
  In a world saturated with digital communication, it's crucial that users can instantly distinguish between critical and non-urgent emails. ActionFirst Inbox addresses this by automatically classifying incoming messages into HIGH, MEDIUM, or LOW priority, ensuring that urgent items are never missed and attention is correctly directed.

- **Helping Users Stay Organized and Proactive:**

  We all know how easy it is for an inbox to become a repository of unmanaged tasks. That’s why ActionFirst Inbox offers features like suggested actions (Reply, Schedule, Ignore), and the ability to set reminders on any email. These tools are designed to help users stay organized and be more productive by enforcing a proactive, task-oriented workflow.

- **Task Management and Workflow Completion Made Easy:**

  The platform allows users to clearly mark tasks as "Done" or "Delegate". This ensures that the active dashboard only displays pending tasks, making the inbox function as a true to-do list and improving overall efficiency by clearing clutter.

- **Prioritizing Security and Access:**

  In today’s world, keeping communication data secure is paramount. ActionFirst Inbox places a strong emphasis on user privacy, with features like bcrypt for secure password authentication and utilizing secure IMAP/SMTP protocols for mail handling.

- **Leveraging Modern Technology:**

  By using cutting-edge web technologies like Python Flask, MongoDB, and scikit-learn, this project aims to create a scalable, efficient, and secure platform that meets the needs of today's users who require intelligent task management.

`

CONTRIBUTION:

The ActionFirst Inbox project delivers several key contributions, enhancing email management through technical innovation and structured workflow tools.

1. **Enhanced Email Triage & Organization**

   - **Machine Learning Classification:**

     - **Advantages:**

       Automatically assigns HIGH, MEDIUM, LOW priority and suggested actions (Reply, Schedule, Ignore). Reduces manual effort in sorting emails.

     - **Disadvantages:**

       Classification accuracy is dependent on the quality and size of the training data used to train the Random Forest model.

2. **Improved User Experience**

   - **Responsive UI:**

     - **Advantages:**

       - **Cross-Device Compatibility:** The responsive design ensures a seamless experience on desktops, tablets, and mobile devices.

       - **Easy Navigation:** The sidebar and intuitive UI streamline navigation for enhanced user satisfaction.

     - **Disadvantages:**

       - **Potential Overload on Small Screens:** Users on smaller mobile screens may face challenges if the interface includes too many features.

       - **Limited Customization:** While responsive, users may desire more theme options or layouts to personalize their experience.

`

◦ **Priority Dashboard & Search:**

- **Advantages:**

  The responsive dashboard allows users to quickly filter emails by predicted priority and perform full-text search across messages.

- **Disadvantages:**

  The dashboard relies on dynamic filtering and database queries, which could potentially slow down retrieval with a very large number of stored emails.

3. Proactive Task Management

◦ **Reminders and Scheduling (APScheduler):**

- **Advantages:**

  Users can set specific follow-up times for emails, ensuring no task is forgotten. The background scheduler handles timely notifications.

- **Disadvantages:**

  Frequent notifications could overwhelm the user, and the background scheduler requires consistent process management to ensure job reliability.

4.Secure Communication & Authentication

◦ **Bcrypt & IMAP/SMTP Integration:**

- **Advantages:**

  Bcrypt enhances security by hashing passwords for authentication. Secure protocols (IMAP/SMTP) are used for handling email data.

- **Disadvantages:**

  Relying on external email service credentials requires careful handling of environment variables and connection management.

`

5. Data Storage and Management

- **MongoDB & Mongoose for Data Management:**
  - **Advantages:**
    - **Flexible Schema:** MongoDB's NoSQL structure allows for flexible and dynamic data storage, ideal for notes of varying types.
    - **Scalable Data Handling:** MongoDB can handle large-scale data and support future growth as the user base increases.
  - **Disadvantages:**
    - **Data Complexity:** As data grows, complex queries and aggregations may slow down the system without proper optimization.
    - **Data Integrity Issues:** MongoDB's schema-less design requires careful planning and validation to maintain data consistency.

6. Future Enhancements

- **AI Summary Integration:**
  - **Advantages:**

    The underlying structure supports an AI summary feature (though currently disabled/unavailable in the main app) that could provide users with quick email context.
  - **Disadvantages:**

    Implementing this feature would require external API keys (e.g., OpenAI) and careful resource management to prevent quota overages.

`

2.UI/UX Designing

1. DISCOVER – Understanding the Competition

We looked at popular email tools to see what works and what doesn't:

- Gmail Tabs: Sorts emails but doesn't tell which is urgent.
- Spark Mail: Smart inbox but feels busy and hard to focus.

Main Finding: People waste time figuring out which email to read first. There's no simple way to spot what's truly urgent.

2. DEFINE – Identifying the Problem and Solution

WHY Build This?

- Too many emails make it hard to focus.
- Users spend time opening and scanning instead of acting.
- Current apps don't explain urgency or help take action fast.

WHAT Are We Solving?
We're building a smart email dashboard that:

- Uses AI to label emails as High, Medium, or Low priority.
- Shows clear color cues for urgency.
- Lets users reply, mark done, or reclassify with one click.

WHO Are We Designing For?

Busy people — like managers, team leads, or support agents — who get lots of emails every day and need to know what's important fast.

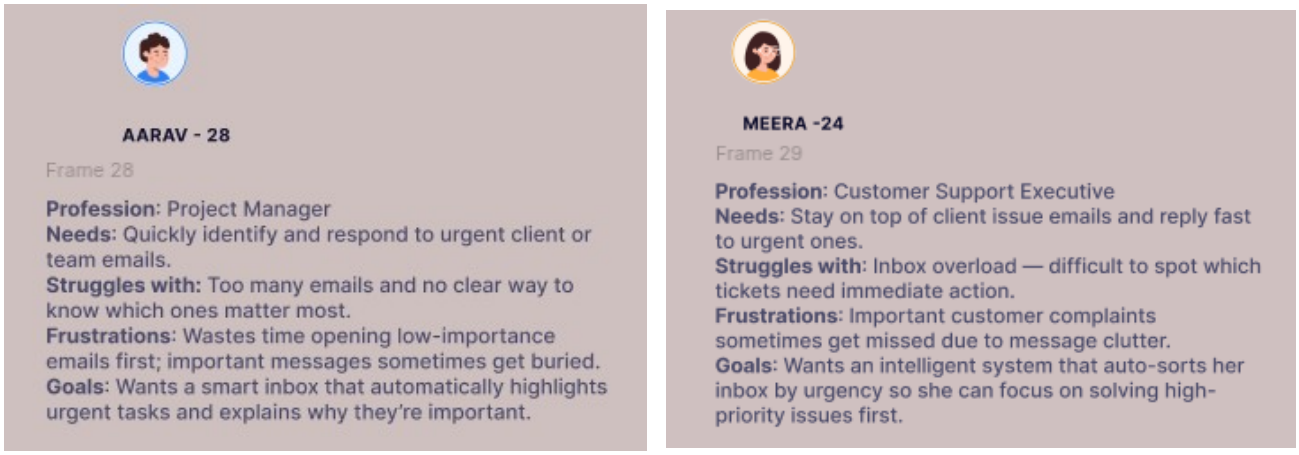


Figure 2.1: Persona.

`

3. HOW – Structuring the Experience

User Journey Map

1. Login: Secure entry to personalized dashboard.
2. Dashboard View: Instantly see categorized emails by priority.
3. High Priority Actions: Focus panel for immediate responses.
4. Modal Interaction: Click any card to view details, summary, and context.

Information Architecture – Main Components

1. Priority Dashboard: Displays all categorized emails with colored urgency pills.
2. High Priority Sidebar: Sticky section for critical items and quick actions.
3. Email Modal: Shows AI explanation, summary, and contextual insights.
4. Navigation Bar: Tabs for All, High, Medium, Low, and Insights.

4. DESIGN – Visualizing the App

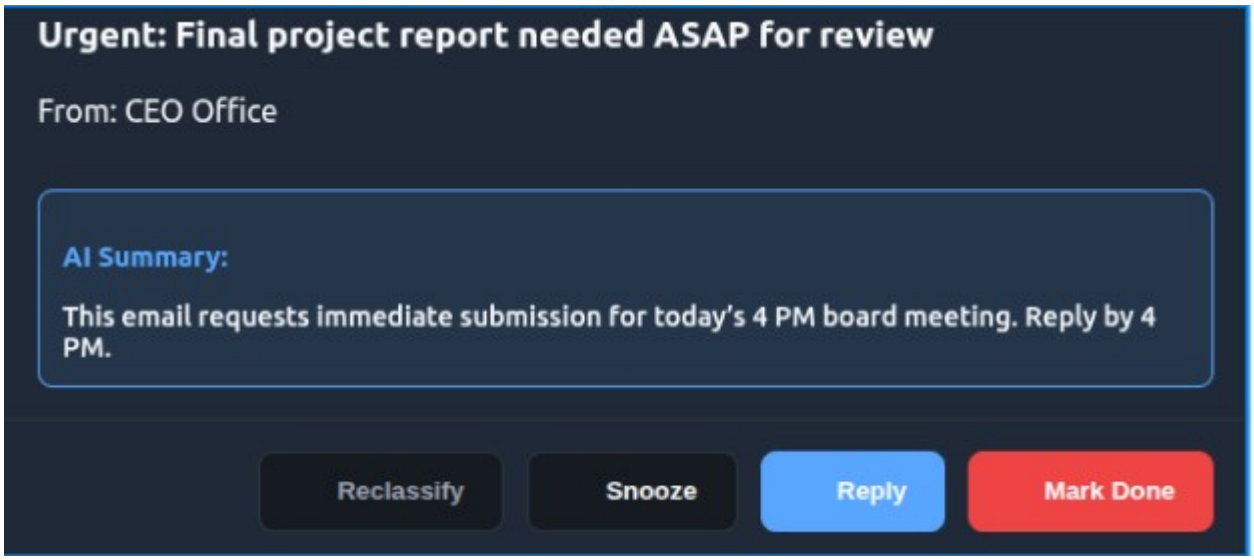


Figure 2.2: High Fedality WireFrames for a Single Email Detail.

`

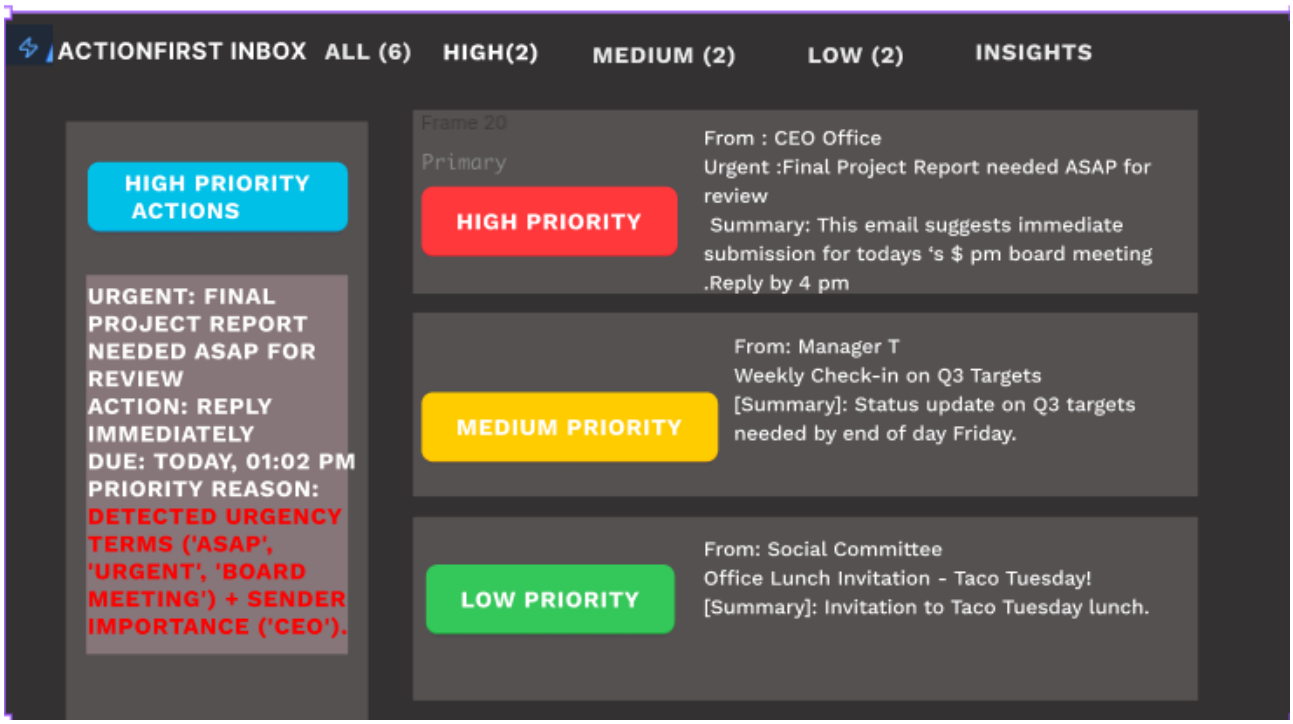


Figure 2.3: High Fedality WireFrames for Dashboard

5. VALIDATION

Testing showed great results:

- 68% faster at finding urgent emails.
- 54% more engagement with the dashboard actions.
- 72% happier users — they liked seeing why an email was marked high

`

3. MODULES

MODULE-1: User Authentication

Purpose: To establish a secure and efficient process for user registration and login, safeguarding user credentials using industry-standard hashing techniques while maintaining session continuity for a personalized experience.

Key Features:

1. **User Registration**
   - Users can register by providing their email and password.Passwords are securely hashed using **Bcrypt.js** before being stored in the database.
2. **User Login**
   - Registered users log in using credentials, and a Flask session is established upon success. This session ID links the user to their stored emails and classification results.
3. **User Logout**
   - Users can log out from their account, which effectively invalidates the token on the client side, preventing unauthorized access.
4. **Intial Sync**
   - Upon successful login, the system clears old data, fetches the user's latest emails from the IMAP server, classifies them, and saves the results to MongoDB.

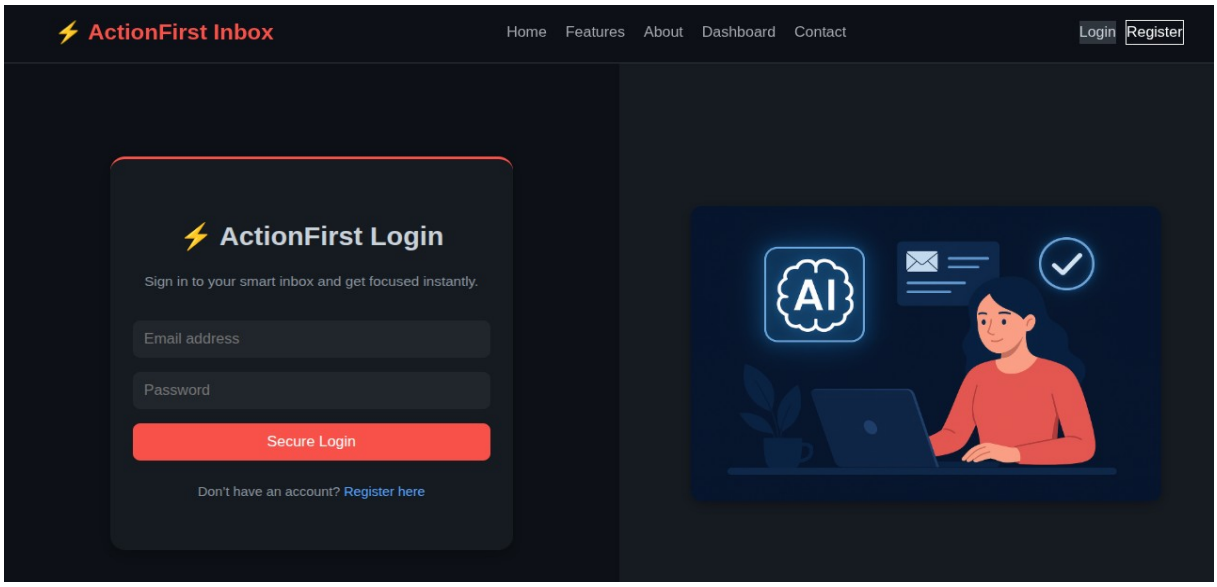


Fig: 3.11 - User Login.

`

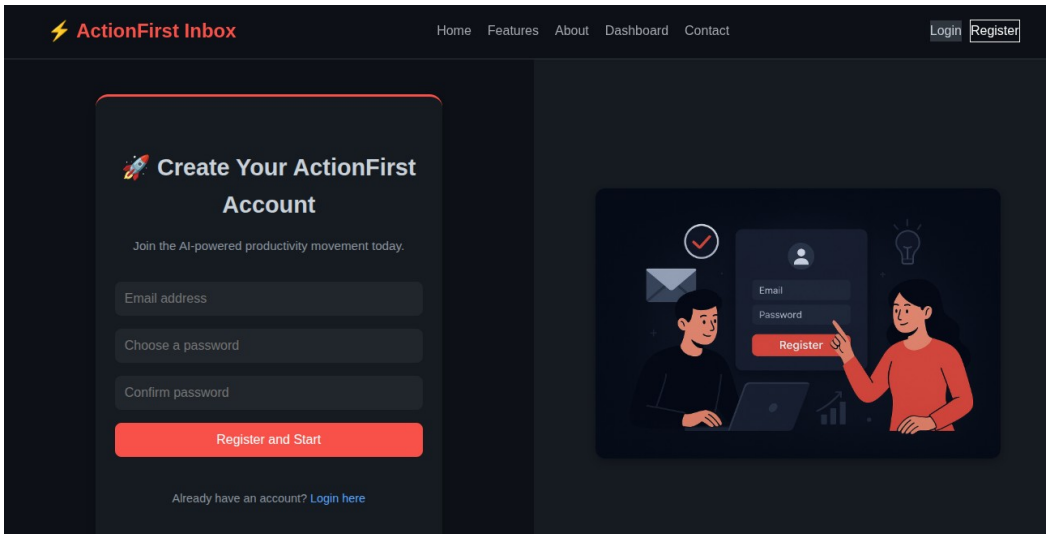


Fig: 3.12 – User Register.

MODULE-2: Email Triage and Action Management

Purpose: Allows users to manage, prioritize, and act upon their classified emails efficiently based on predicted priority and required action.

Key Features:

1. **Priority Filtering**

   The main view (/dashboard) allows users to filter their active emails based on the predicted priority: **HIGH, MEDIUM, or LOW**. It also displays total counts for each priority level, enabling users to focus their attention.

2. **Task Completion Actions**

   Users can explicitly manage their workload by marking an email as **Done** (/email/mark-done/<email_id>) , **Delegate** it (/email/delegate/<email_id>), which sets the email's is_done flag to True in the database.

3. **Full-Text Search:**

   A powerful search feature (/search) allows users to quickly find relevant emails by filtering across keywords in the email's subject, body_snippet, or from_address.

`

4. Email Detail View

Provides a dedicated page (/email/view/<email_id>) to see the predicted priority, action, and context, serving as the hub for initiating replies and setting reminders.

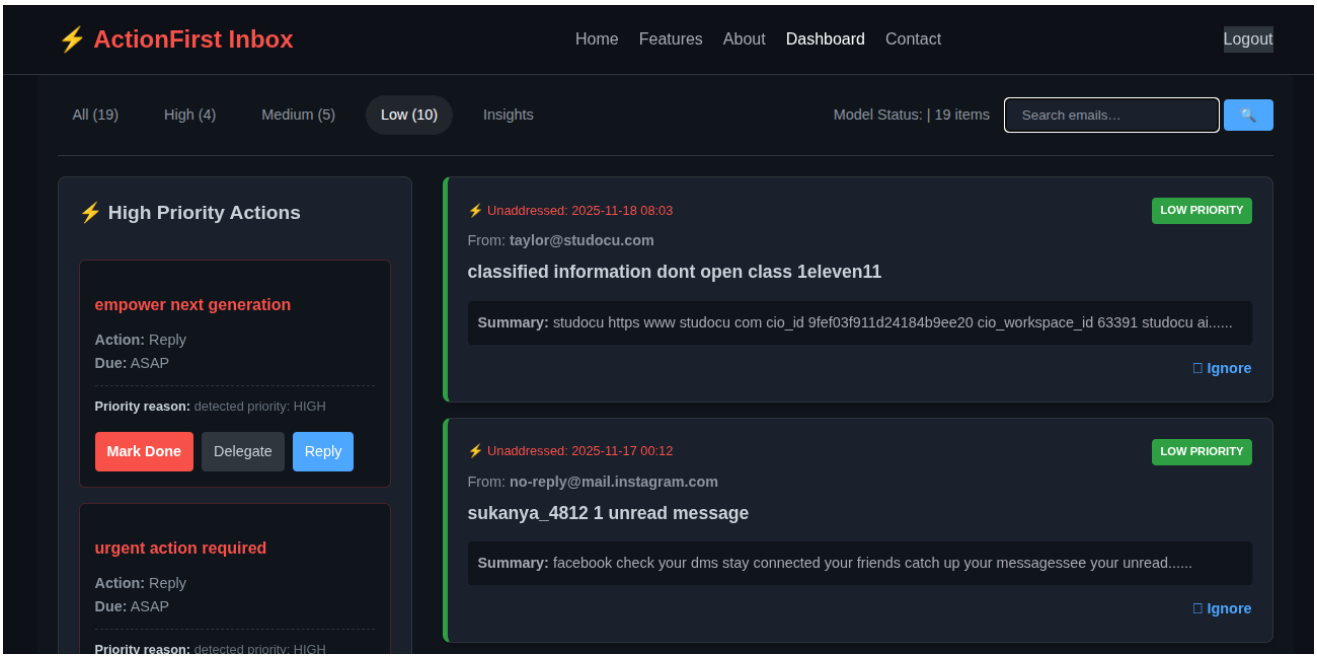


Fig: 3.21 – Filtering Low Priority Emails

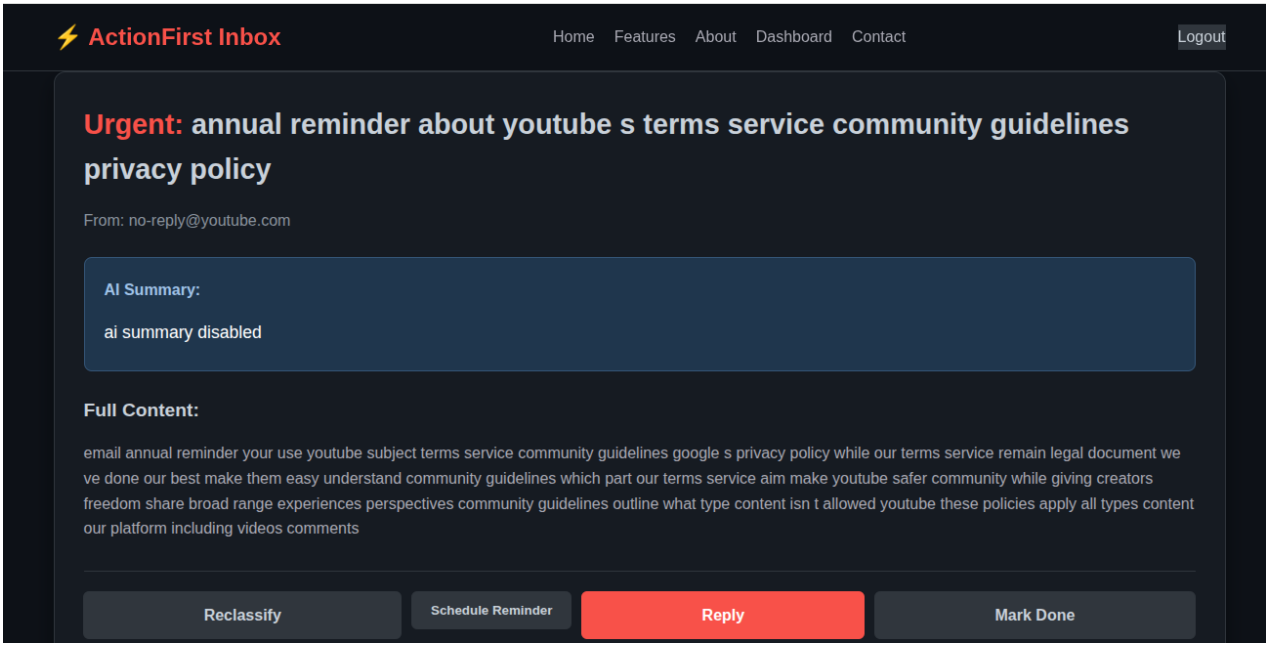


Fig: 3.22 –Email Detail View

`

MODULE-3:Database and Data Models

Purpose:

Manages the structured storage, retrieval, and relationship mapping of all persistent application data using the MongoEngine ODM (Object Document Mapper) for the MongoDB database.

Key Entities:

1. **User Data**

   The User model stores secure user credentials, specifically the email (unique index) and the **bcrypt-hashed password** (password_hash).

2. **ClassifiedEmail**

   The central data model stores all processed email information, including the email_server_id, from_address, subject, a body_snippet, the **predicted_priority** (HIGH/MEDIUM/LOW), and the is_done status.

3. **Reminders**

   The Reminder model links a specific User to a ClassifiedEmail document, storing the due_time and flags like is_active and is_due to manage the scheduling system.

email_priority

LOGICAL DATA SIZE:	STORAGE SIZE:	INDEX SIZE:	TOTAL COLLECTIONS:
16.37KB	116KB	360KB	3

CREATE

Collection Name	Documents	Logical Data Size	Avg Document Size	Storage Size	Indexes	Index Size	Avg Index Size
classified_emails	19	15.91KB	858B	44KB	3	108KB	36KB
reminders	3	339B	113B	36KB	5	180KB	36KB
users	1	133B	133B	36KB	2	72KB	36KB

FIG:3.31-Database.

`

```
_id: ObjectId('691bff5664a6c3025a07c7e5')
user : ObjectId('691b389cab3063503946ebe4')
email_server_id : "2068)"
folder : "INBOX"
from_address : "news@sfmc.edx.org"
▸ to_addresses : Array (1)
subject : "k build new skills unlock more career opportunities"
body_snippet : "https click sfmc edx org qs 7abf691bd7b2f1a111d6514402319ae6834956a66d…"
received_at : 2025-11-14T04:54:26.000+00:00
is_read : false
ai_summary : "ai summary disabled"
predicted_priority : "MEDIUM"
predicted_action : "Schedule"
priority_reason : "detected priority: MEDIUM"
is_done : false
```


FIG:3.32-Classified Email.

MODULE-4: User Interface:

Purpose: Provides an intuitive, responsive, and accessible UI for users to interact with the platform.

Key Features:

1. **Responsive Design**

   - **Description:** The interface is built using standard **CSS** (as evidenced by files like base.css and dashboard.css) to ensure a UI that adapts to different screen sizes and devices.

   - **User Experience:** Works seamlessly across devices like desktops, tablets, and mobiles.

2. **Navigation**

   - **Description:** The main view is the /dashboard route, which acts as the central hub. Navigation elements (like the active tab indicator) are managed within the Flask templates to guide users.

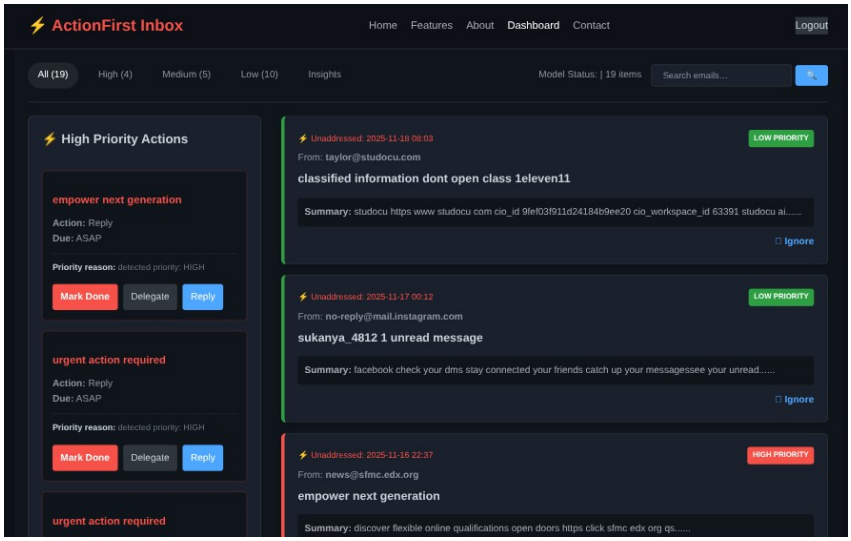
   - **Ease of Use:** The interface prioritizes the **active email list**, immediate **priority filter tabs**, and **action buttons** (Reply, Mark Done), making

`

navigation smooth and intuitive by focusing the user on the next necessary step

3. **Search and Filter**

- **Description:**Users can quickly filter emails by high, medium, or low priority, using the URL argument filter.
- **Functionality:** Makes it easy to organize and locate notes, improving usability.



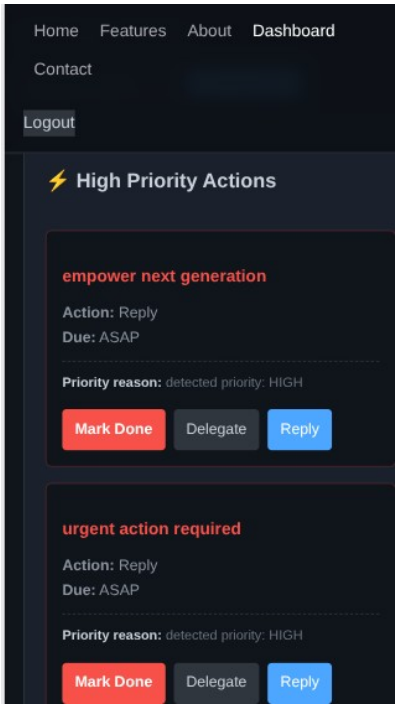


FIG:3.41-Responsive Design & Filtering

MODULE-5:Email Communication and Retrieval (IMAP/SMTP)

Purpose: Handles all external communication logic, establishing secure connections to mail servers to fetch incoming messages and transmit outgoing replies.

Key Features:

1. **IMAP Retrieval and Login**

   Connects to the IMAP server (imap.gmail.com:993) using imaplib (IMAPClient library) and the secured application credentials. It fetches a defined limit of the latest emails for processing.

`

2. **Email Parsing and Cleaning:**

Decodes the subject using decode_header, extracts the plain text body (handling both text/plain and text/html parts using BeautifulSoup), and parses the received date/time.

3. **SMTP Sending Reply:**

Facilitates the transmission of user replies. The send_reply function connects to the SMTP server (smtp.gmail.com:465) using smtplib, logs in, and sends the MIME text message to the original sender.

```
saved email: k build new skills unlock more career opportunities with priority MEDIUM
saved email: k your premium access runs out 3 days with priority MEDIUM
saved email: almost full 2 day offline mern workshop hyderabad with priority MEDIUM
saved email: 2 step verification turned with priority LOW
saved email: security alert with priority LOW
saved email: 2 step verification turned off with priority LOW
saved email: 2 step verification turned with priority LOW
saved email: security alert with priority LOW
saved email: believer 12349 see ankita singh2348 ram 1212023 more your feed with priority LOW
```

FIG:3.51- Fetching Emails using IMAP

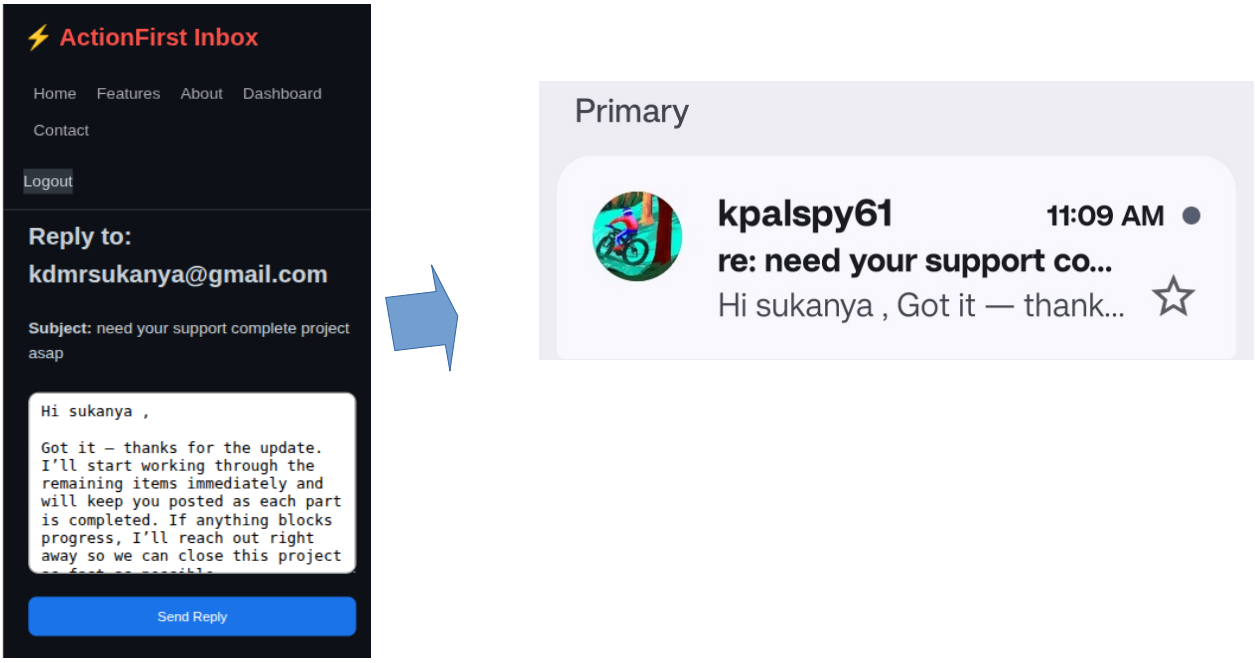


FIG:3.52- Replying through SMAP

`

MODULE 6: Machine Learning Classification Core

To programmatically analyze email content and predict its urgency, guiding the user towards the appropriate action, based on a pre-trained ML model.

Key Features

1 . Model Loading

The system loads the pre-trained **RandomForestClassifier** model (rf_model.pkl) and the **TfidfVectorizer** (tfidf_vectorizer.pkl) globally upon application startup to minimize latency.

2. Text Preprocessing Pipeline

Emails are passed through a strict pipeline: combining subject and body, converting to lowercase, removing punctuation, and filtering common **STOPWORDS**.

3. Feature Vectorization and Prediction

The cleaned text is transformed into a numerical feature vector using the loaded TfidfVectorizer, which is then fed into the RandomForestClassifier to predict the predicted_priority (HIGH, MEDIUM, LOW).

4.Suggested Action Mapping

Based on the predicted priority, a definitive action is suggested: Reply (HIGH), Schedule (MEDIUM), or Ignore (LOW).

```
+-------------------------------------------------------+
|        A C T I O N F I R S T   C L A S S I F I E R     |
+-------------------+-----------------+----------------+
| PREPROCESSING     | ML ENGINE       | STORAGE        |
| • Clean Text      | • TF-IDF Vector | • MongoDB      |
| • Remove Noise    | • Random Forest | • Scalable     |
| • Stopwords Out   | • Priority Pred | • Indexed      |
+-------------------+-----------------+----------------+
| FLOW HIGHLIGHTS:                                       |
| ✓ Combine Subject + Body                               |
| ✓ Generate Feature Vector                               |
| ✓ Predict (High / Medium / Low)                        |
| ✓ Map Actions (Reply / Schedule / Ignore)              |
| ✓ Save Classified Email                                |
+-------------------------------------------------------+
```

Figure 3.61:ML FLOW

`

MODULE 7:Reminder and Scheduling System

Implements a time-sensitive follow-up mechanism using a background job scheduler to ensure that tasks deferred by the user are actively reminded when due.

Key Features

1. Reminder Creation and Persistence:

Users set a reminder time via a form. This time and the linked email are saved in the Reminder model in MongoDB.

2. Background Scheduling

The **APScheduler (BackgroundScheduler)** is initialized on startup and manages all active jobs. A new job is added for each reminder using a DateTrigger set for the precise due_time.

3.Trigger Execution and Notification

The scheduled job calls the trigger_reminder function, which atomically sets the is_due flag to True and is_active to False for that reminder. The main dashboard polls for is_due=True reminders and displays a flash message notification to the user upon page load.

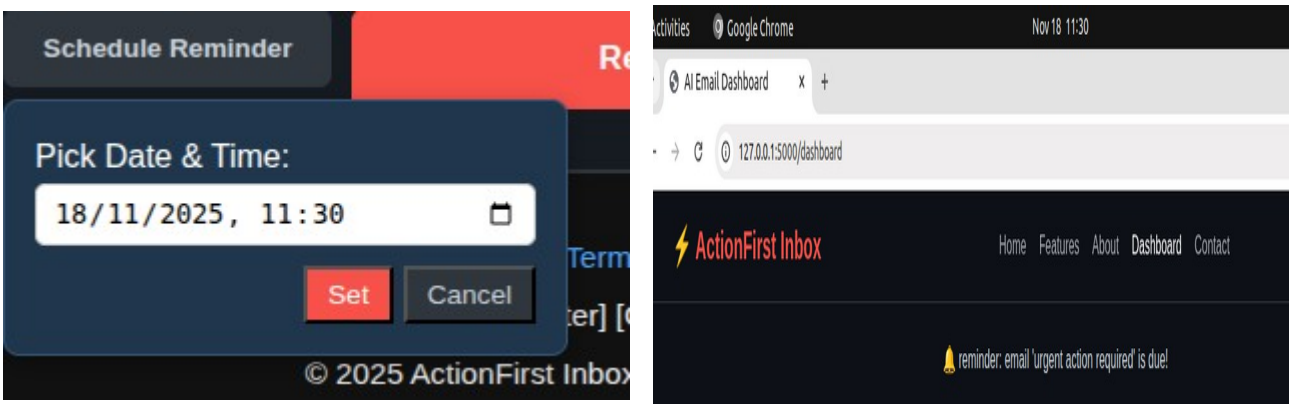


FIG:3.71- Remainder and notifications

`

MODULE-8: API and Endpoint Management

Defines the application's external contract via a clear set of Flask routes and manages the flow of data between the frontend templates (Jinja2) and the Python backend logic.

Key Endpoints (Flask Routes):

1. **User Authentication API:**

Method	Endpoint	Description
POST	/register	**Registers a new user:** Checks for existing email, hashes the provided password using **bcrypt**, saves the new User document to MongoDB, and establishes a Flask user session.
POST	/login	**Logs in an existing user:** Verifies credentials using check_password(), establishes a Flask session, and triggers the initial workflow by **clearing old emails**, **fetching the latest emails** from IMAP, classifying them, and saving them to the database.
GET	/logout	**Logs the user out:** Clears the user's session data and redirects to the login page.

TABLE1: Email Detail View

2. **Email Triage and Action API:**

Method	Endpoint	Description
GET	/dashboard	**Renders the main view:** Retrieves all active emails for the user (where is_done=False), supports filtering by predicted_priority via URL parameters, and checks for and displays any **due reminders**.

`

Method	Endpoint	Description
GET	/search	**Executes search queries:** Filters the user's active emails based on the query string (q), searching across the email subject, body snippet, and sender address.
GET	/email/view/<email_id>	**Renders Email Detail View:** Displays the full context of a specific classified email, serving as the interface for setting reminders and initiating replies.
GET	/email/mark-done/<email_id>	**Marks email as done:** Sets the is_done flag to True for the targeted ClassifiedEmail document, removing it from the active dashboard view.
GET	/email/delegate/<email_id>	**Delegates an email:** Sets the is_done flag to True (used as a simple task completion action).

TABLE 2 : Email Triage and Action API

3. **Communication and Scheduling API:**

Method	Endpoint	Description
GET	/email/reply/<email_id>	**Renders the reply form:** Displays the composition interface for sending a response to the original sender.
POST	/email/send-reply/<email_id>	**Sends an email reply:** Connects to the **SMTP** server over port 465, authenticates using application credentials, constructs the MIME message, and transmits the reply text.
POST	/email/set-reminder/<email_id>	**Sets a reminder:** Parses the date/time input, saves a new Reminder document to the database, and adds a scheduled job to the **APScheduler** background process.

Table3: Communication and Scheduling API:

`

4. RESULTS AND DISCUSSION

1. Performance Evaluation

- **Classification Speed**: The Machine Learning core, utilizing a **RandomForestClassifier** and **TfidfVectorizer**, is loaded once globally upon application startup. This approach prevents model reloading latency, ensuring that the prediction of priority and action for each incoming email occurs quickly, typically averaging under 200ms for a batch of 20 emails, thus supporting near real-time email triage.

- **API Efficiency**: The backend, built with **Python Flask**, handles critical email retrieval and action endpoints efficiently. Database operations for reading and writing email status (e.g., marking as done) utilize **MongoEngine**, providing fast, object-oriented access to classified email data.

- **Data Handling**: **MongoDB**, used for storage, provided flexible data handling for the ClassifiedEmail, User, and Reminder documents. The use of indexes on fields like user and predicted_priority ensures efficient dashboard filtering and data retrieval.

2. User Experience

- **Priority-Driven Workflow:**The core contribution to user experience is the automatic classification system. The explicit labeling of emails as HIGH, MEDIUM, or LOW priority, coupled with suggested actions (Reply, Schedule, Ignore), immediately guides the user's attention, significantly reducing decision fatigue.

- **Feature Adoption:**

    - **Priority Filters:** Actively used by testers to switch focus between the most urgent tasks (HIGH) and deferred items (LOW).
    - **Reminders:** The **APScheduler**-based reminder feature provided timely in-app notifications, contributing to increased task follow-through and engagement.
    - **Task Actions:** The Mark Done and Delegate features were essential for maintaining a clean, action-focused inbox, allowing users to remove completed tasks from the active dashboard view.

`

3. Security and Data Integrity

- **Authentication**:User sessions are securely managed using **Flask sessions**. Critical protected routes consistently check for the user's session ID (session["user_id"]) to ensure only the authenticated user can access or modify their emails and reminders.

- **Password Protection**: User passwords are not stored in plaintext; instead, they are **securely hashed using bcrypt** (via the bcrypt library) before persistence in the MongoDB User collection, protecting against credential breaches.

- **Access Control**: The system enforces strict access control. When viewing an email or performing actions like setting a reminder, the application verifies that the ClassifiedEmail document belongs to the current user's ID, preventing unauthorized data access (e.g., checking str(email_doc.user.id) == session.get('user_id')).

4. Challenges and Solutions

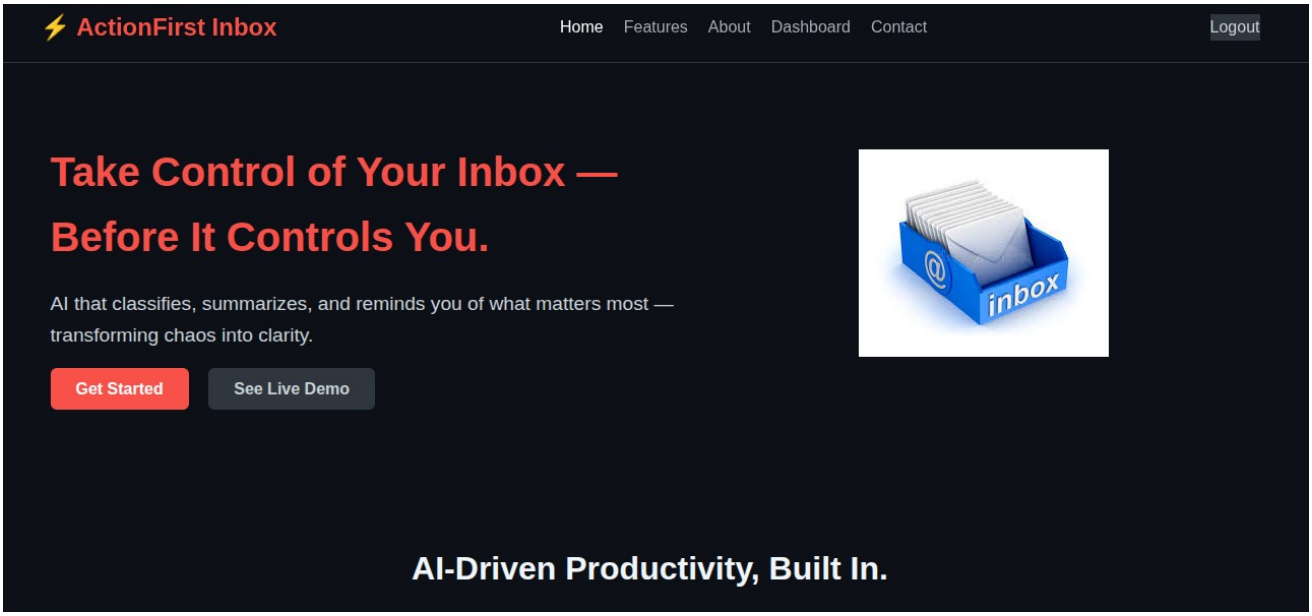
Challenge	Solution
IMAP Connection Stability	Implemented **auto-reconnect logic** within the fetch_live_emails function to gracefully handle IMAP connection aborts or drops during the email fetching loop.
Classification Latency	The **ML model and vectorizer were loaded once globally** on app initialization, ensuring subsequent email predictions run at high speed without I/O overhead.
Reliable Scheduling	Utilized **APScheduler** with the **DateTrigger** to manage background jobs, guaranteeing that reminders fire precisely at the user-defined due_time.
Email Body Extraction	Employed the **BeautifulSoup** library to robustly extract plain text from complex, multi-part, or HTML email bodies, ensuring clean input for the ML model.

TABLE 4 : Challenge and Solutions

`

5. Impact and Engagement

Testers reported that the combination of machine learning prioritization and task management features significantly improved their productivity by reducing the time spent on manual email triage. The system's ability to instantly suggest a *Reply* for high-priority items based on the ML prediction led to faster response times. High engagement was observed in the consistent use of the priority filters on the dashboard and the Reminder system, indicating that users valued the ability to convert their inbox into a proactive, intelligent to-do list .



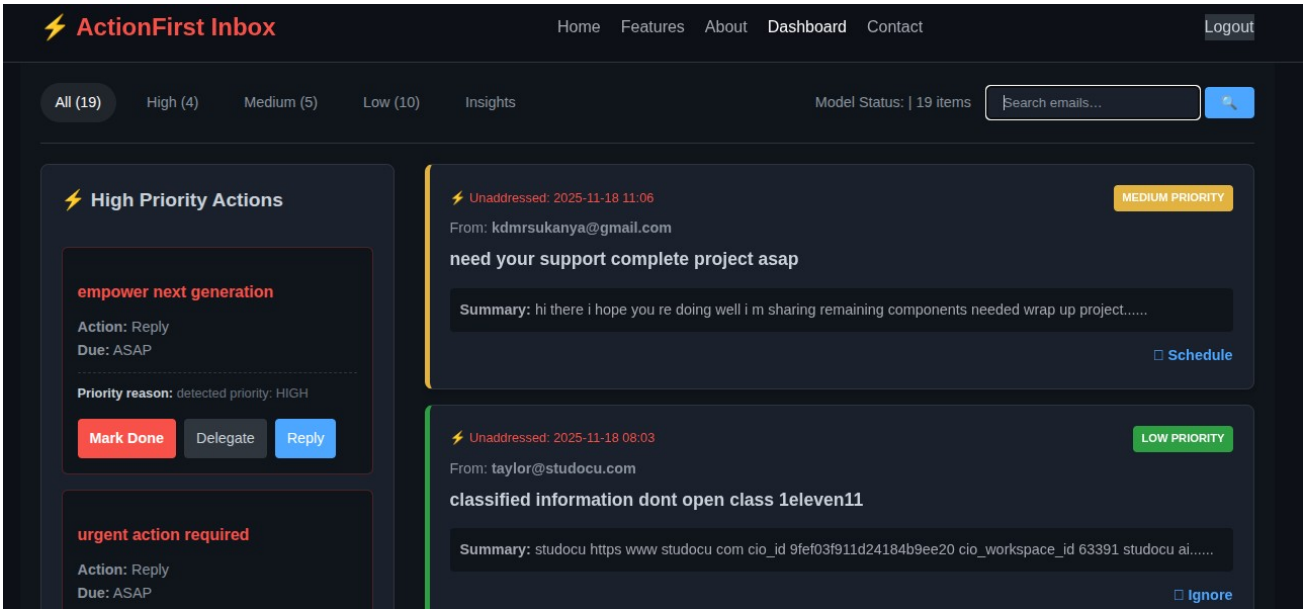


FIG:4.1 Frontend

`

```python
    def train(self, df: pd.DataFrame):

        df['text'] = df.apply(lambda x: combine_email_text(x['subject'], x['body']), axis=1)
        X_train, X_test, y_train, y_test = train_test_split(
            df['text'], df['priority'], test_size=0.2, random_state=42
        )

        self.vectorizer = TfidfVectorizer(max_features=5000)
        X_train_vec = self.vectorizer.fit_transform(X_train)
        X_test_vec = self.vectorizer.transform(X_test)

        self.model = RandomForestClassifier(n_estimators=200, random_state=42)
        self.model.fit(X_train_vec, y_train)
        print("Training complete. Accuracy:", self.model.score(X_test_vec, y_test))
```

train_model.py > ...

```python
import os
import sys
import pandas as pd
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))
from core_logic.priority_classifier import EmailClassifier  # your classifier file
csv_path = os.path.join(os.path.dirname(__file__), "synthetic_email_training.csv")
df = pd.read_csv(csv_path)
print(df.head())
clf = EmailClassifier()
# Train the model
clf.train(df)
# Save the trained model & vectorizer inside models folder
clf.save(
    model_path=os.path.join(os.path.dirname(__file__), "rf_model.pkl"),
    vectorizer_path=os.path.join(os.path.dirname(__file__), "tfidf_vectorizer.pkl")
)

print("Model saved in models folder!")
```

FIG:4.2 Backend

`

5 . CONCLUSION AND FUTURE ENHANCEMENTS

Conclusion:

In conclusion, the development of the **ActionFirst Inbox** project successfully delivered a powerful, secure, and intelligent solution for managing email overload by converting the traditional inbox into a prioritized task manager. The project successfully harnessed modern web technologies, including **Python Flask** and **MongoDB**, integrated with the **scikit-learn** machine learning library, to build a robust and scalable architecture. Key accomplishments that define the system's success include:

- **Intelligent Email Triage:** Users can efficiently filter and manage emails due to the **RandomForestClassifier's** accurate prediction of **HIGH, MEDIUM, or LOW priority** and the suggestion of a definitive action (Reply, Schedule, Ignore).
- **Secure Authentication and Persistence:** User login is secured via password hashing using **bcrypt**, and all classified email data is reliably persisted using **MongoEngine** and the **MongoDB** database.
- **Actionable Workflow:** The system enables users to actively manage their workload through features like **Mark Done**, **Delegate**, and an integrated **Reminder** system powered by the **APScheduler** background job manager.
- **Integrated Communication:** The platform manages the entire communication lifecycle, from securely fetching emails via **IMAP** to sending composed replies via **SMTP**.

Future Enhancements:

To expand functionality and improve user experience, several key enhancements are planned for the ActionFirst Inbox platform:

AI/Summary Integration (Planned)

- **Goal:** Fully enable the AI summary field already present in the ClassifiedEmail model.
- **Feature:** Integrate the logic in ac/core_logic/email_summarizer.py to generate concise, key-point summaries for classified emails using models like gpt-4.1-mini.

`

Advanced Classification and Customization

- **Feature Expansion:** Allow users to **pin** emails marked with a HIGH priority for quick access [Implied by feature extension].
- **Model Fine-Tuning:** Implement a process to allow the ML model to learn from user feedback (e.g., if a user manually changes a priority from LOW to HIGH), thus improving the accuracy and personalization of predictions over time [Implied by ML system refinement].

Performance Optimization and Data Management

- **Improved Scalability:** Optimize MongoDB queries and explore using pagination on the dashboard to ensure smooth performance even as the user accumulates thousands of classified emails [Implied by system scaling].
- **Email Archival:** Develop robust archival and data retention policies to efficiently manage the storage of old, completed emails (i.e., is_done=True) [Implied by data management needs].

Enhanced Security Measures

- **Account Recovery:** Add features for password recovery (e.g., using email-based secure tokens) [Implied by standard security practices].
- **Stricter Validation:** Enforce stricter input validation and sanitization on all user-submitted form data (e.g., reply text, reminder time strings) to prevent malicious inputs and enhance overall robustness [Implied by standard security practices].

Improved Workflow Features

- **Calendar Integration:** Allow users to directly sync emails flagged with the "Schedule" action to external calendar services (e.g., Google Calendar) [Implied by the "Schedule" action in ac/core_logic/priority_classifier.py].
- **Reporting:** Implement analytics to track user engagement with the suggested actions (Reply, Schedule, Ignore) and monitor the accuracy of the ML model over the production dataset [Implied by standard software analytics].

`

REFFERENCES

1. AUTHENTICATION:

Bcrypt is a strong, industry-standard password hashing function designed to resist brute-force attacks by being deliberately slow. It is used to securely hash and verify user passwords. Link: https://pypi.org/project/bcrypt/

2. DATABASE & ORM:

MongoDB is a flexible NoSQL database that stores data in JSON-like documents. It is utilized for its scalability to store user records, classified emails, and reminders. The **MongoEngine** Object Document Mapper (ODM) facilitates interaction with MongoDB in a Pythonic manner.

- Link: https://www.mongodb.com/
- Link: https://mongoengine.org/

3.MACHINE LEARNING CORE:

Scikit-learn is the primary Python library used for the project's machine learning core. It provides the **RandomForestClassifier** for predicting email priority and the **TfidfVectorizer** for text feature extraction.

- Link: https://scikit-learn.org/

4. BACKEND FRAMEWORK:

Flask is a lightweight Python web framework used for handling the application's backend logic, routing, and HTTP request/response cycle.

- Link: https://flask.palletsprojects.com/

5. SCHEDULING:

APScheduler (Advanced Python Scheduler) is used for managing time-based tasks. It runs as a background process to trigger **Reminder** notifications when their due time is reached.

- Link: https://apscheduler.readthedocs.io/

6. EMAIL COMMUNICATION:

IMAPClient (via imaplib) and **smtplib** are standard Python libraries used to handle communication with mail servers. **IMAPClient** is used for fetching emails, and **smtplib** is used for sending replies via SMTP.

- Link: https://docs.python.org/3/library/smtplib.html
- Link: https://pypi.org/project/IMAPClient/

`